

Open in app ↗

Sign up

Sign in

Medium

 Search Write

Deployment of Machine learning Models

Jose Armando 

Follow

4 min read · Apr 18, 2023



9



Here we will create a model with pycaret in Google Collab, and we will deploy it in a Web App with Flask and publish a docker image and Web Application in Azure.

1. Train and save a model with Google Collab.

```
# save pipeline/model  
save_model(lr, model_name = 'short_deployment_28042020')
```

<https://github.com/armandoordonez/mlops/blob/main/MLOps.ipynb>

Get Jose Armando's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

2. Download the trained model (short_deployment_28042020)

The screenshot shows a Jupyter notebook interface with a file explorer on the left and a code editor on the right. The file explorer shows a directory with files: sample_data, logs.log, and short_deployment_28042020.pkl. The code editor shows a table of data and a code cell for saving the model.

	age	bmi	children	age^2	age bmi	age children	bmi^2	bmi children	children^2
300	-0.220584	-0.521212	1.577352	-0.383008	-0.415391	1.260325	-0.569778	1.274050	1.408662
904	1.483407	0.757039	-0.879223	1.661025	1.735095	-0.829425	0.689659	-0.852059	-0.586055
670	-0.646582	0.159393	1.577352	-0.734326	-0.501665	0.912033	0.063025	1.584265	1.408662
617	0.702411	-0.851356	0.758494	0.597312	0.091446	1.066829	-0.845738	0.465023	0.300486
373	-0.930581	0.384569	0.758494	-0.933051	-0.678652	0.176751	0.291332	0.840597	0.300486

```
[16]
# save pipeline/model
save_model(lr, model_name = 'short_deployment_28042020')

Transformation Pipeline and Model Successfully Saved
(Pipeline(memory=FastMemory(location=/tmp/joblib),
  steps=[('numerical_imputer',
    TransformerWrapper(include=['age', 'bmi', 'children'],
      transformer=SimpleImputer()),
    ('categorical_imputer',
      TransformerWrapper(include=[],
        transformer=SimpleImputer(strategy='most_frequent'))),
    ('polynomial_features',
      TransformerWrapper(transformer=PolynomialFeatures(include_bias=False))),
    ('normalize', TransformerWrapper(transformer=StandardScaler())),
    ('trained_model', LinearRegression(n_jobs=-1))]),
  'short_deployment_28042020.pkl')
```

3. Create a Flask app locally (For example in Visual Studio Code)

```
from flask import Flask, request, url_for, redirect, render_template, jsonify
from pycaret.regression import *
import pandas as pd
import pickle
import numpy as np

app = Flask(__name__)

model = load_model('deployment_28042020')
cols = ['age', 'sex', 'bmi', 'children', 'smoker', 'region']

@app.route('/')
def home():
    return render_template("home.html")

@app.route('/predict', methods=['POST'])
def predict():
```

```

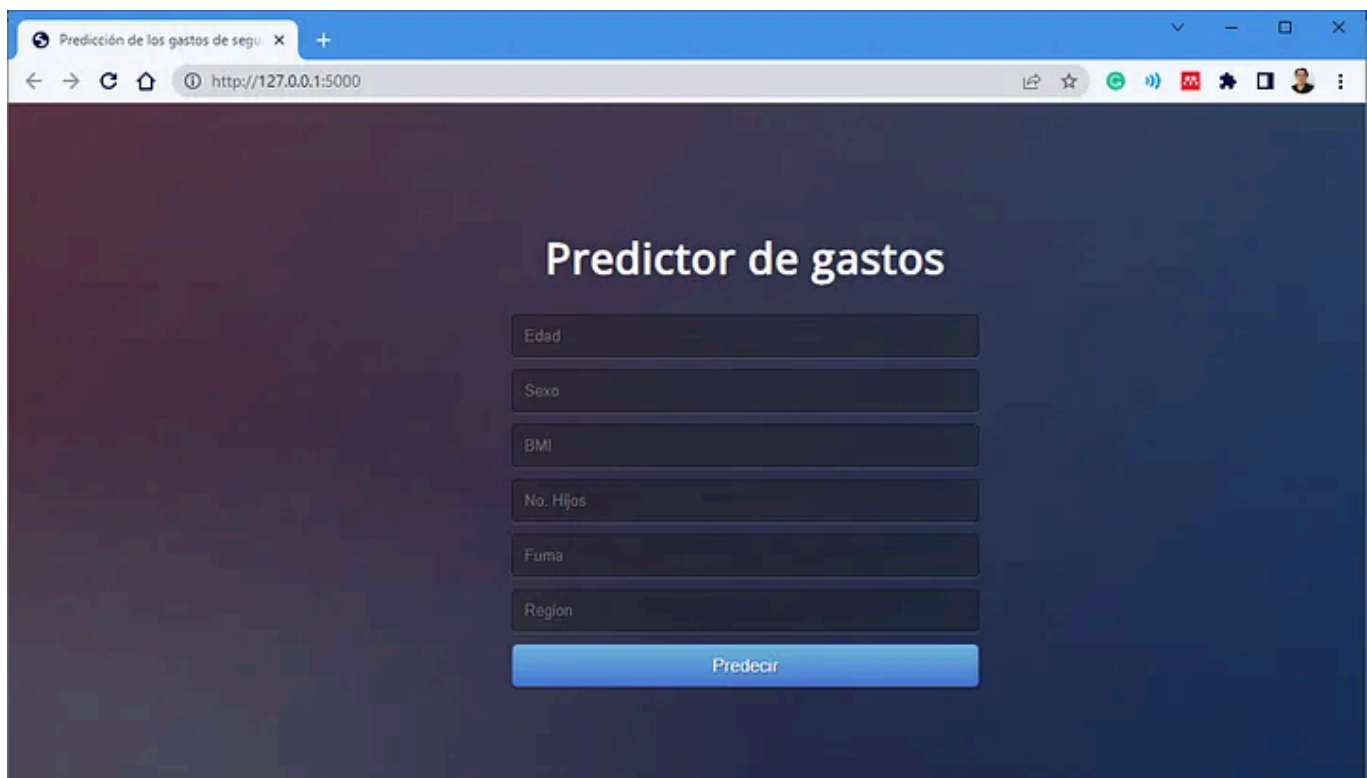
int_features = [x for x in request.form.values()]
final = np.array(int_features)
data_unseen = pd.DataFrame([final], columns = cols)
prediction = predict_model(model, data=data_unseen, round = 0)
# prediction = 1540000
#print("*****",prediction)
prediction = int(prediction.prediction_label[0])
return render_template('home.html',pred='Los gastos esperados son {}'.format

@app.route('/predict_api',methods=['POST'])
def predict_api():
    data = request.get_json(force=True)
    data_unseen = pd.DataFrame([data])
    prediction = predict_model(model, data=data_unseen)
    #prediction = 1540000
    output = prediction.Label[0]
    return jsonify(output)

if __name__ == '__main__':
    app.run(debug=True)

```

4. Run the application locally: <http://127.0.0.1:5000/>



The screenshot shows a web browser window with the title "Predicción de los gastos de segu...". The address bar shows the URL "http://127.0.0.1:5000". The main content area has a dark blue background with the title "Predictor de gastos" in white. Below the title, there are six input fields stacked vertically, each with a label: "Edad", "Sexo", "BMI", "No. Hijos", "Fuma", and "Region". At the bottom of the form is a blue button labeled "Predecir".

5. Modify the code to make it light: just for academic purposes we will make a light version of the flask server, this will ease or deployment process.

```
from flask import Flask,request, url_for, redirect, render_template, jsonify
# from pycaret.regression import *
# import pandas as pd
# import pickle
# import numpy as np
import os

app = Flask(__name__)

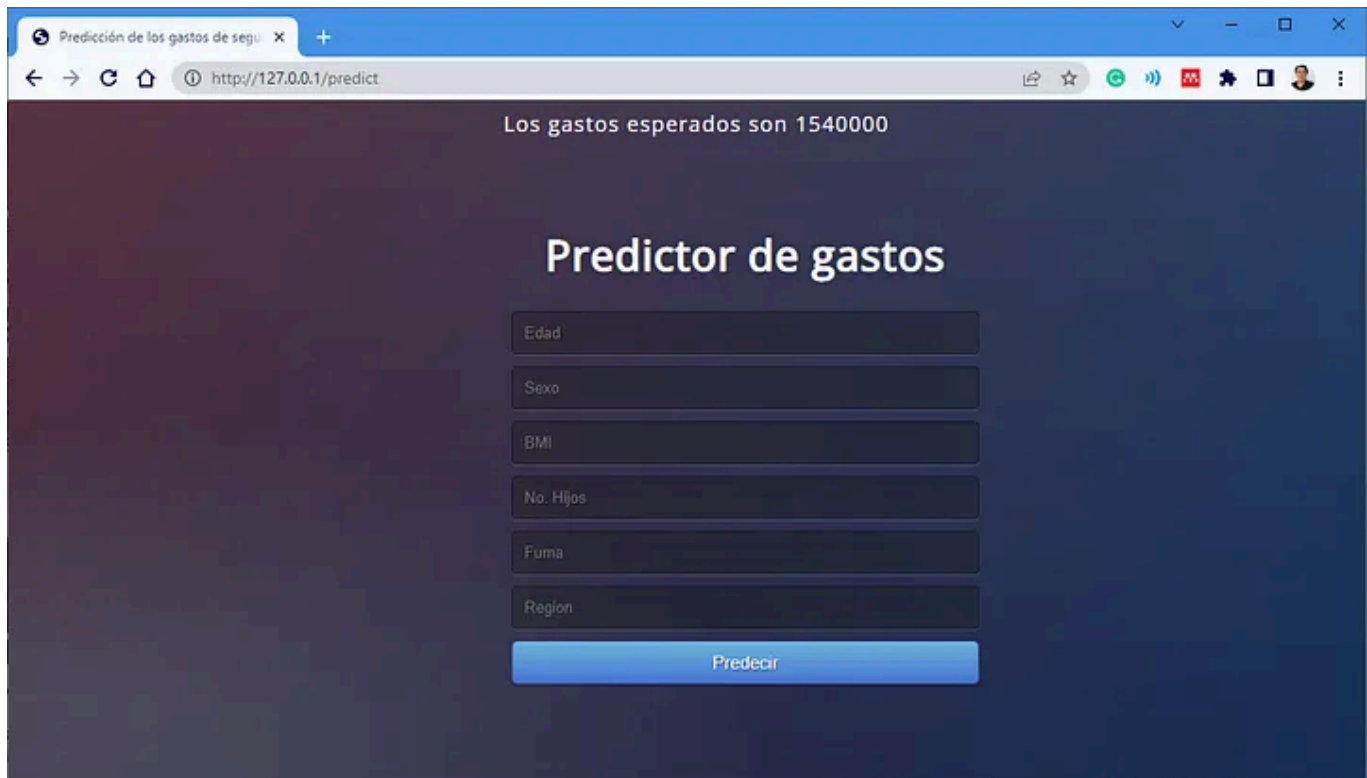
# model = load_model('deployment_28042020')
cols = ['age', 'sex', 'bmi', 'children', 'smoker', 'region']

@app.route('/')
def home():
    return render_template("home.html")

@app.route('/predict',methods=['POST', 'GET'])
def predict():
    #int_features = [x for x in request.form.values()]
    #final = np.array(int_features)
    #data_unseen = pd.DataFrame([final], columns = cols)
    # prediction = predict_model(model, data=data_unseen, round = 0)
    prediction = 1540000
    #print("*****",prediction)
    # prediction = int(prediction.prediction_label[0])
    return render_template('home.html',pred='Los gastos esperados son {}'.format(prediction))

@app.route('/predict_api',methods=['POST'])
def predict_api():
    #data = request.get_json(force=True)
    #data_unseen = pd.DataFrame([data])
    #prediction = predict_model(model, data=data_unseen)
    prediction = 1540000
    #output = prediction.Label[0]
    return jsonify(prediction)

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 80))
    app.run(debug=True, host='0.0.0.0', port=port)
```



6. Create a file called Dockerfile (without extensions)

```
# start by pulling the python image
FROM python:3.8-alpine

RUN pip install --upgrade pip
RUN pip install virtualenv

ENV VIRTUAL_ENV=/venv
RUN virtualenv venv -p python3

ENV PATH="VIRTUAL_ENV/bin:$PATH"

WORKDIR /app
ADD . /app

# install dependencies
RUN pip install -r requirements.txt

# expose port
EXPOSE 80

# configure the container to run in an executed manner
ENTRYPOINT [ "python" ]
```

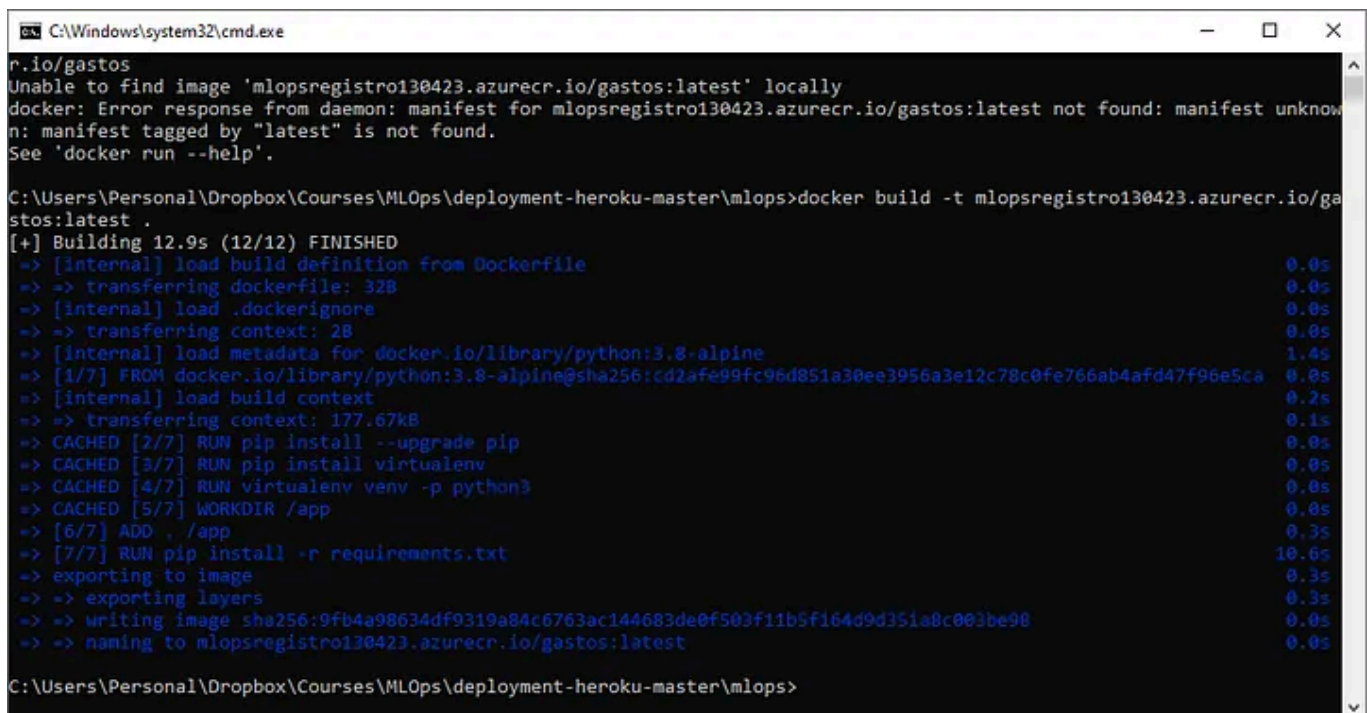
```
CMD ["app_light.py" ]

# run application
# CMD ["python", "app_light.py"]
```

8. Test the container locally (You need a docker installation in your own machine)

```
cd Dropbox\Courses\MLOps\deployment-heroku-master\mlops

docker build -t mlopsregistro130423.azurecr.io/gastos:latest .
```



```
C:\Windows\system32\cmd.exe
r.io/gastos
Unable to find image 'mlopsregistro130423.azurecr.io/gastos:latest' locally
docker: Error response from daemon: manifest for mlopsregistro130423.azurecr.io/gastos:latest not found: manifest unknown: manifest tagged by "latest" is not found.
See 'docker run --help'.

C:\Users\Personal\Dropbox\Courses\MLOps\deployment-heroku-master\mlops>docker build -t mlopsregistro130423.azurecr.io/gastos:latest .
[+] Building 12.9s (12/12) FINISHED
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring dockerfile: 32B 0.0s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 2B 0.0s
=> [internal] load metadata for docker.io/library/python:3.8-alpine 1.4s
=> [1/7] FROM docker.io/library/python:3.8-alpine@sha256:cd2afe99fc96d851a30ee3956a3e12c78c0fe766ab4afd47f96e5ca 0.0s
=> [internal] load build context 0.2s
=> => transferring context: 177.67kB 0.1s
=> CACHED [2/7] RUN pip install --upgrade pip 0.0s
=> CACHED [3/7] RUN pip install virtualenv 0.0s
=> CACHED [4/7] RUN virtualenv venv -p python3 0.0s
=> CACHED [5/7] WORKDIR /app 0.0s
=> [6/7] ADD . /app 0.3s
=> [7/7] RUN pip install -r requirements.txt 10.6s
=> exporting to image 0.3s
=> => exporting layers 0.3s
=> => writing image sha256:9fb4a98634df9319a84c6763ac144683de0f503f11b5f164d9d351a8c003be98 0.0s
=> => naming to mlopsregistro130423.azurecr.io/gastos:latest 0.0s

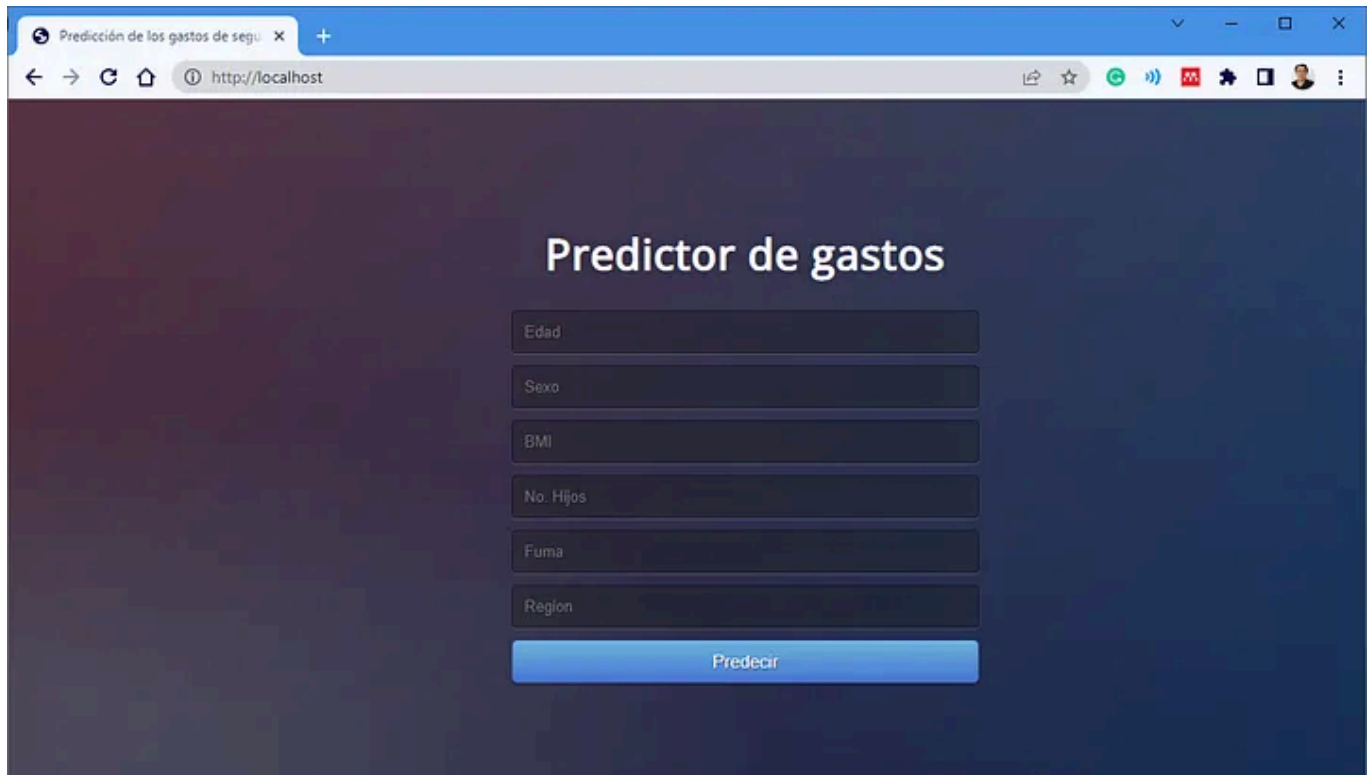
C:\Users\Personal\Dropbox\Courses\MLOps\deployment-heroku-master\mlops>
```

```
docker run -d -p 80:80 mlopsregistro130423.azurecr.io/gastos
```

```
C:\Windows\system32\cmd.exe

C:\Users\Personal\Dropbox\Courses\MLOps\deployment-heroku-master\mlops>docker run -d -p 80:80 mlopsregistro130423.azurecr.io/gastos
897ea815604e68c401805b3f23c8a2a5d6eb0224570b1e528a73396020f6341b

C:\Users\Personal\Dropbox\Courses\MLOps\deployment-heroku-master\mlops>
```



9. Create a docker repository in Azure

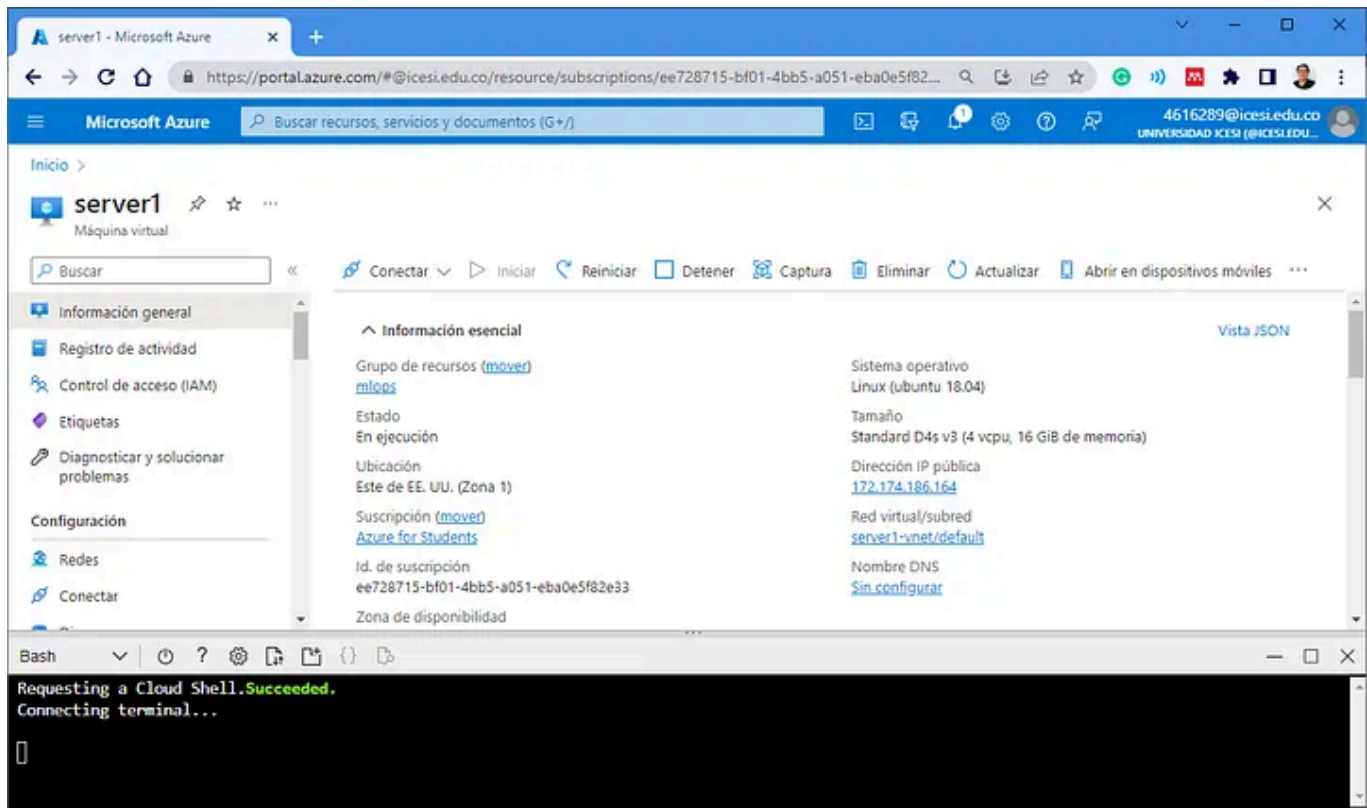
The screenshot shows the Azure portal interface for a container registry. The left sidebar contains navigation options like 'Información general', 'Registro de actividad', 'Control de acceso (IAM)', 'Etiquetas', 'Inicio rápido', 'Eventos', 'Configuración', 'Claves de acceso', 'Cifrado', 'Identidad', 'Redes', 'Microsoft Defender for Cloud', and 'Búsqueda'. The main content area displays 'Información esencial' for the registry 'mlopsregistro130423'. It includes details such as the resource group 'mlops', location 'Este de EE. UU.', subscription 'Azure for Students', and a list of SKUs (Standard, Standard, Standard). Below this, there are usage statistics: 'Uso' showing 100 GiB included, 0.05 GiB used, and 0.00 GiB storage. A section for 'ACR Tasks' is also visible, describing the task runner capabilities.

10. Create a virtual machine

The screenshot shows the Azure portal interface for a virtual machine named 'server1'. The left sidebar contains navigation options like 'Información general', 'Registro de actividad', 'Control de acceso (IAM)', 'Etiquetas', 'Diagnosticar y solucionar problemas', 'Configuración', 'Redes', 'Conectar', 'Discos', 'Tamaño', 'Microsoft Defender for Cloud', 'Recomendaciones de Advisor', and 'Extensiones + aplicaciones'. The main content area displays 'Información esencial' for the VM 'server1'. It includes details such as the resource group 'mlops', location 'Este de EE. UU. (Zona 1)', subscription 'Azure for Students', and a list of SKUs (Standard D4s v3). Below this, there are network and storage details, including the public IP address '172.174.186.164' and the virtual network 'server1-vnet/default'. A section for 'Propiedades' (Properties) is also visible, showing the operating system as Linux and the size as Standard D4s v3 (4 vcpu, 16 GiB de memoria).

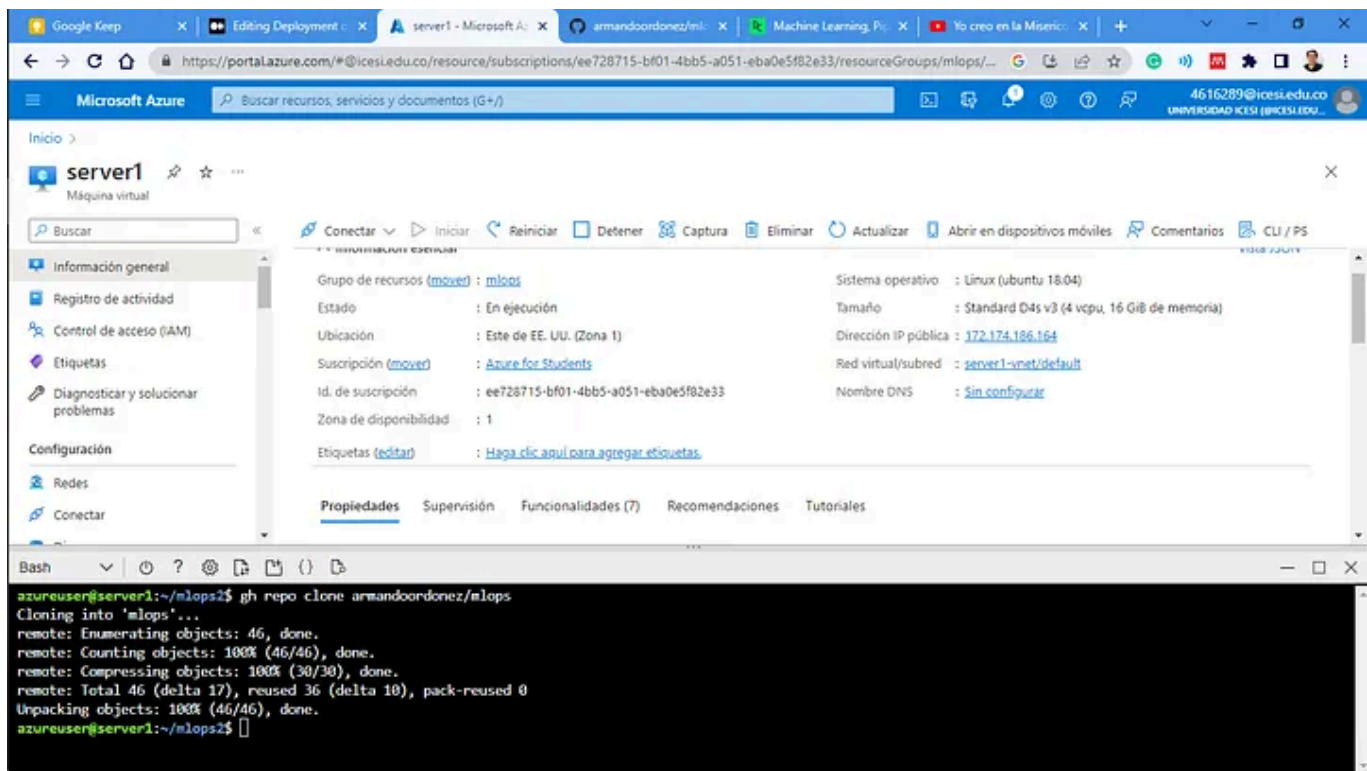
11. Connect to the machine

```
ssh -i server1_key.pem azureuser@172.174.186.164
```



12. Clone the code from github

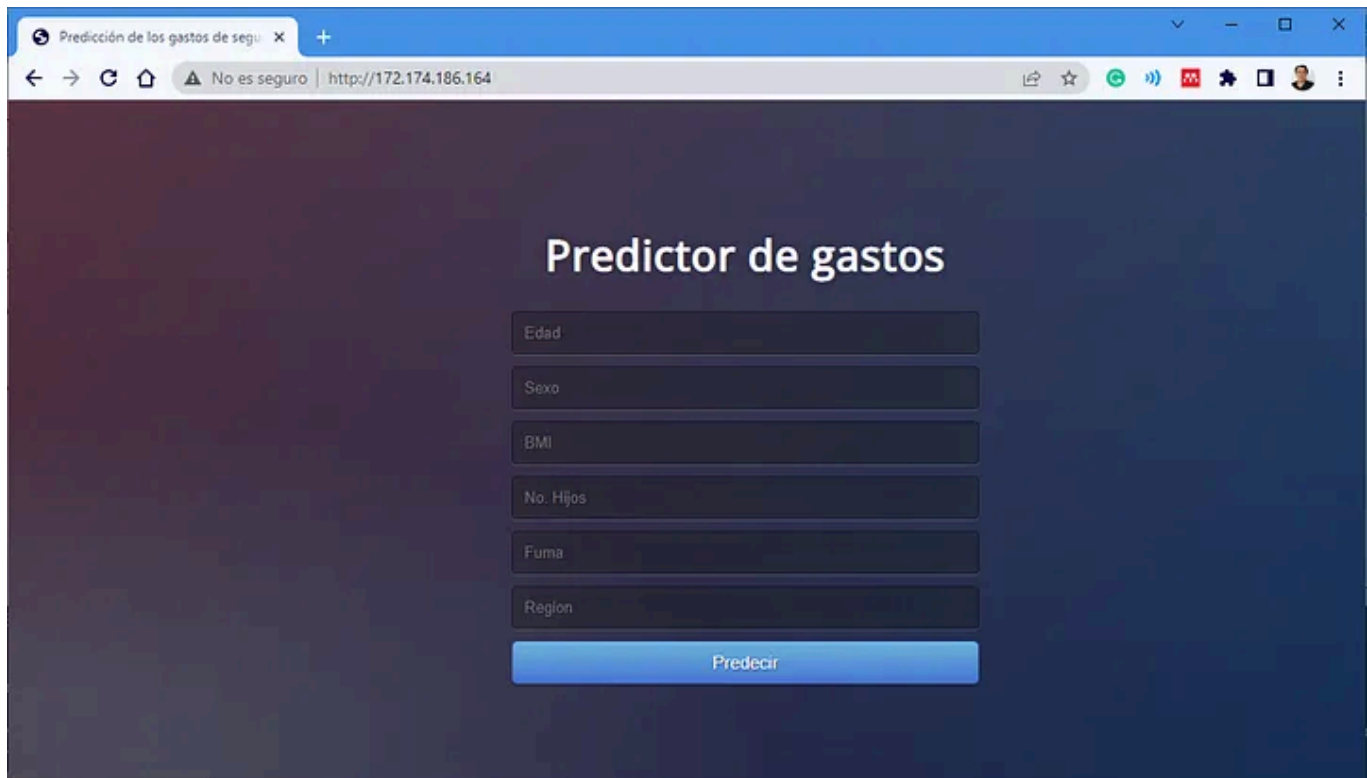
```
gh repo clone armandoordonez/mlops
```



13. Create an image and a container in the new server using the previous commands.

```
azureuser@server1:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS        NAMES
azureuser@server1:~$ docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED
mlpsregistro130423.azurecr.io/costos   latest      bd2fbf26fc5b     3 days ago
ubuntu              latest      08d22c0ceb15     5 weeks ago
hello-world         latest      feb5d9fea6a5     19 months ago
azureuser@server1:~$
```

14. Try the app on the virtual server



Predicción de los gastos de segu...

No es seguro | http://172.174.186.164

Predicador de gastos

Edad

Sexo

BMI

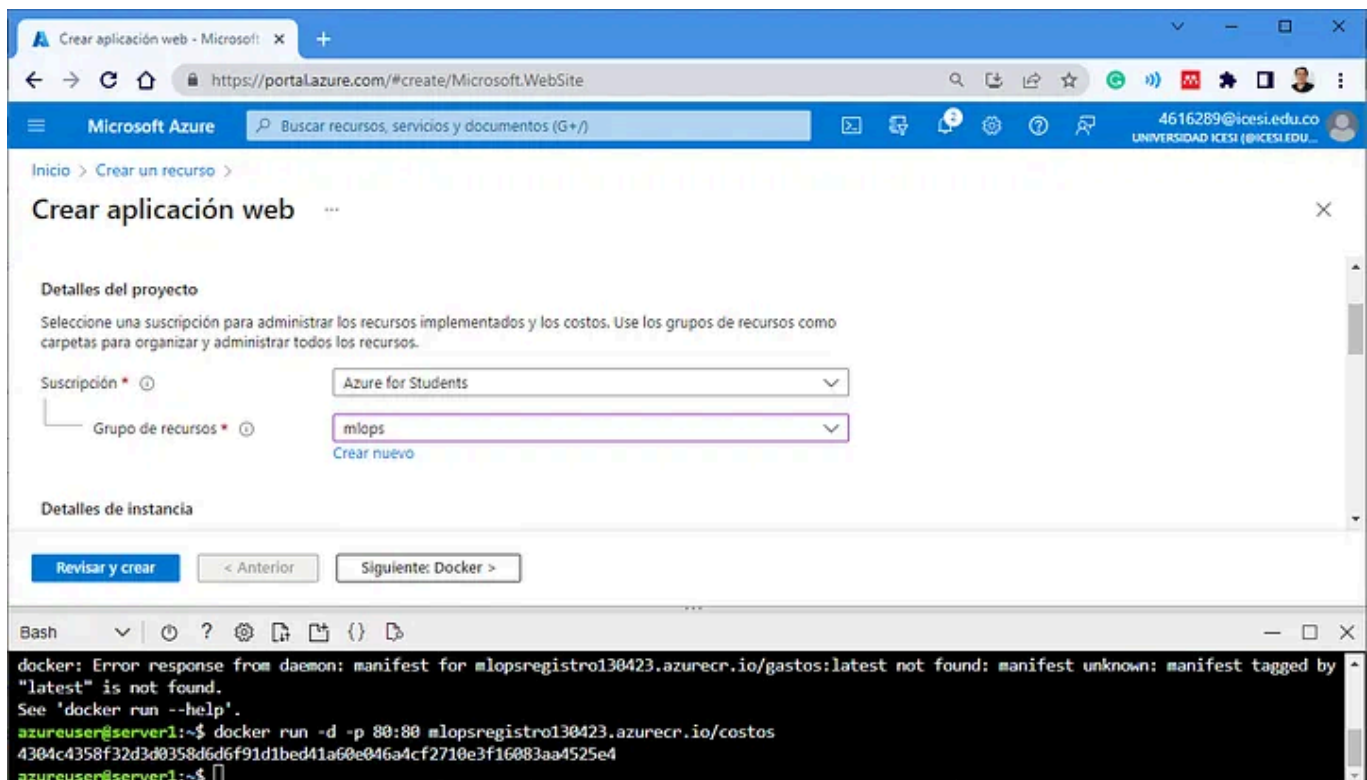
No. Hijos

Fuma

Region

Predecir

15. Create a Web App from docker



Crear aplicación web - Microsoft

https://portal.azure.com/#create/Microsoft.WebSite

Microsoft Azure

Buscar recursos, servicios y documentos (G+)

4616289@icesi.edu.co

UNIVERSIDAD ICESI (@ICESI.EDU...)

Inicio > Crear un recurso >

Crear aplicación web

Detalles del proyecto

Seleccione una suscripción para administrar los recursos implementados y los costos. Use los grupos de recursos como carpetas para organizar y administrar todos los recursos.

Suscripción * Azure for Students

Grupo de recursos * mlops

Crear nuevo

Detalles de instancia

Revisar y crear

< Anterior

Siguiente: Docker >

Bash

```
docker: Error response from daemon: manifest for mlopsregistro130423.azurecr.io/gastos:latest not found: manifest unknown: manifest tagged by "latest" is not found.
See 'docker run --help'.
azureuser@server1:~$ docker run -d -p 80:80 mlopsregistro130423.azurecr.io/gastos
4304c4358f32d3d0358d6d6f91d1bed41a60e046a4cf2710e3f16083aa4525e4
azureuser@server1:~$
```

The screenshot shows the 'Crear aplicación web' (Create web application) page in the Microsoft Azure portal, specifically the 'Docker' tab. The page is for creating a new web application. The 'Opciones' (Options) section shows 'Contenedor único' (Single container) selected. The 'Origen de imagen' (Image source) is 'Azure Container Registry'. Under 'Opciones de Azure Container Registry' (Azure Container Registry options), the 'Registro' (Registry) is 'mlopsregistro130423', the 'Imagen' (Image) is 'costos', and the 'Etiqueta' (Tag) is 'latest'. The 'Comando de inicio' (Startup command) field is empty. At the bottom, there are buttons for 'Revisar y crear' (Review and create), '< Anterior' (Previous), and 'Siguiendo: Redes >' (Next: Networks).

The screenshot shows the 'calculocostos' web application overview page in the Microsoft Azure portal. The page displays various details about the application, including its group, state, location, subscription, and implementation details. The 'Propiedades' (Properties) tab is selected, showing the application's name, domain, plan, operating system, and status. The 'Implementación' (Implementation) section shows the application is running on a Linux operating system. The 'Configuración' (Configuration) section shows the application is configured for authentication and application insights.

Essentials

- Grupo de recursos (mover): mlops
- Estado: Running
- Ubicación (mover): Este de EE. UU.
- Suscripción (mover): Azure for Students
- Id. de suscripción: ee728715-bf01-4bb5-a051-eba0e5f82e33
- Etiquetas (editar): Haga clic aquí para agregar etiquetas.

Propiedades

- Nombre: calculocostos
- Dominio predeterminado: calculocostos.azurewebsites.net
- Plan del servicio de aplicaciones: ASP-mlops-8274 (B1: 0)
- Sistema operativo: Linux
- Comprobación de estado: Sin configurar

Implementación

- Implementación: Aplicación web

Configuración

- Configuración: Configuración
- Autenticación: Autenticación
- Application Insights: Application Insights

12. Test the Web Application.

Predicción de los gastos de seguro

https://calculocostos.azurewebsites.net/predict

Los gastos esperados son 1540000

Predictor de gastos

36

male

1000

3

yes

northeast

Predecir

References

<https://www.datacamp.com/tutorial/tutorial-machine-learning-pipelines-mlops-deployment>

Mlops

Mlops Without Much Ops



Written by Jose Armando



11 followers · 1 following

AI Practitioner

Follow



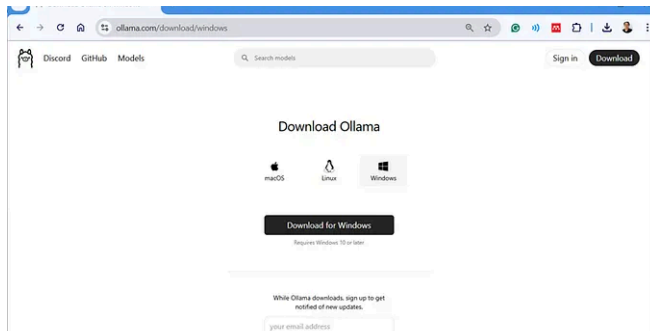
No responses yet



Write a response

What are your thoughts?

More from Jose Armando



 Jose Armando 

Instalación de deepseek de forma local en Windows en 3 pasos!

A continuación, como instalar el modelo deepseek de forma local

Jan 28  1



 Jose Armando 

Análisis y preparación de datos con IA Generati

En este tutorial, aprenderás a manejar valores perdidos, corregir formatos y transformar...

Feb 10





Jose Armando

Creación de flujos inteligentes en 2025 con Make y Gemini

¿Te imaginas que cada vez que recibes un archivo en Google Drive, una IA lo lea,...

Apr 2



Jose Armando

Agente en Copilot: Learning Coach de Ciencia de Datos

Objetivo del taller:

Oct 15

[See all from Jose Armando](#)

Recommended from Medium



In Artificial Intelligence in Plain Eng... by Indie Qua...



Abonia Sojasingarayar

Building MLOps Pipelines That Scale: Lessons from Production...

Most ML failures happen in production. Learn why MLOps & real-world reliability matter...



Nov 8



17



Deploying a Scalable Machine Learning Service on Kubernetes

A Comprehensive Hands-On Tutorial

Aug 18



13



1



Abhay singh

From Machine Learning Models to Production Pipelines with MLOps

Introduction to MLOps

Dec 3



4



Rohan Dutt

How to Build MLOps Pipelines That Detect Data Drift Before It Becom...

Monitoring Techniques to Catch Model Decay Early and Keep Your Predictions Aligned wit...



Nov 10



4



In Activated Thinker by Shreyas Naphad

LangChain: Beginner-Friendly Guide

The simplest breakdown of LangChain you'll ever read



In AI Advances by Ran Tao

ML Feature Engineering, Automated: The LLM Pipeline Tha...

Good features are vital for machine learning success, but manual feature engineering is...



4d ago



150



1



6d ago



225



See more recommendations